

FACADE

JOSHUA ABRIL

PATRONES DE SOFTWARE

UNINPAHU

2025

1) ¿Qué es el Patrón de Diseño Facade?:

El **patrón de diseño *Facade* (Fachada)** es un patrón estructural que proporciona una interfaz unificada y simplificada para un conjunto de interfaces en un subsistema. Su objetivo principal es reducir la complejidad del sistema y ocultar los detalles internos del mismo al ofrecer un punto de acceso único.

2) ¿Cuál es el Propósito Principal del Patrón Facade?:

El propósito principal del patrón de diseño **Facade** es proporcionar una interfaz simplificada y de alto nivel para acceder a un conjunto de interfaces o subsistemas complejos.

3) ¿En que Tipo de Situaciones es Recomendable Usar el Patrón Facade?:

Es Recomendable usar en Situaciones como Trabajar con un Sistema Complejo o Legado, para desacoplar un cliente del sistema, Cuando se necesita una interfaz clara para capas superiores, para agrupar operaciones comunes y para crear una **API** para un subsistema.

4) Una Ventaja de Utilizar el Patrón Facade:

Simplifica el uso de un sistema complejo al proporcionar una interfaz unificada y fácil de usar; ya que esto permite que los desarrolladores interactúen con el sistema sin tener que conocer todos sus detalles internos, lo que reduce la curva de aprendizaje, mejora la legibilidad del código y facilita el mantenimiento.

5) ¿Cómo se relaciona el patrón Facade con el principio de menor conocimiento?:

El patrón **Facade** se relaciona directamente con el principio de menor conocimiento (también conocido como *principio de Ley de Deméter*), porque reduce la cantidad de clases con las que un objeto necesita interactuar directamente; ya que este Principio dice que: "Un objeto solo debería hablar con sus amigos cercanos y no con extraños."

6) ¿Qué elementos componen el patrón Facade?:

Se compone de: Fachada (**Facade**) que es la clase principal del patrón y proporciona una interfaz simple y unificada para que el cliente acceda al subsistema, Subsistemas (**Subsystem Classes**) que son las clases reales que contienen la lógica y funcionalidad del sistema; y el Cliente (Client) que es el usuario del sistema que interactúa con la fachada.

7) ¿Cómo implementar el patrón Facade en un sistema complejo?:

Consiste en crear una clase que simplifique el acceso a múltiples clases internas del sistema, ocultando los detalles complejos y ofreciendo una interfaz clara y directa para el cliente donde primero se identifica el subsistema complejo, después se diseña la clase **Facade**, luego se encapsula la complejidad interna y por último el cliente usa solo la fachada.

8) ¿Qué diferencias existen entre Facade y un Adapter?:

Mientras el Patrón **Facade** simplifica el acceso a un sistema complejo, un **Adapter** hace compatible una interfaz existente con otra diferente; el **Facade** cuenta con Abstracción de alto nivel que proporciona una nueva interfaz simple y el **Adapter** con Abstracción de bajo nivel que convierte una interfaz en otra. Por último, la Interfaz Original de **Facade** no cambia la interfaz del subsistema y la de **Adapter** adapta una interfaz existente para que el cliente la entienda.

9) ¿Cómo contribuye el patrón Facade a la mantenibilidad del código?:

El patrón **Facade** contribuye significativamente a la mantenibilidad del código de varias maneras clave como reduciendo el acoplamiento, centralizando el acceso al Sistema, ocultando la complejidad, facilitando pruebas y depuración; y promoviendo la Separación de Responsabilidades.

10) Ejemplo Práctico donde usar el patrón Facade:

- **Ejemplo:** Un sistema para encender una computadora, que normalmente involucra varios pasos: encender energía, iniciar BIOS, cargar sistema operativo, mostrar escritorio.

- **Código:**

```
class FuenteDePoder {  
    void encender() {  
        System.out.println("Fuente de poder encendida.");  
    }  
}
```

```
class BIOS {  
    void ejecutar() {  
        System.out.println("BIOS ejecutada.");  
    }  
}
```

```
class SistemaOperativo {
```

```
void cargar() {  
    System.out.println("Sistema operativo cargado.");  
}  
}
```

```
class Escritorio {  
    void mostrar() {  
        System.out.println("Escritorio listo.");  
    }  
}
```

```
class ComputadoraFacade {  
    private FuenteDePoder fuente;  
    private BIOS bios;  
    private SistemaOperativo so;  
    private Escritorio escritorio;  
  
    public ComputadoraFacade() {  
        fuente = new FuenteDePoder();  
        bios = new BIOS();
```

```
        so = new SistemaOperativo();  
        escritorio = new Escritorio();  
    }  
  
    public void iniciarComputadora() {  
        fuente.encender();  
        bios.ejecutar();  
        so.cargar();  
        escritorio.mostrar();  
    }  
}  
  
public class Cliente {  
    public static void main(String[] args) {  
        ComputadoraFacade miPC = new ComputadoraFacade();  
        miPC.iniciarComputadora();  
    }  
}
```

- **Algoritmo:**

[Cliente]

|

v

[Facade] --> [Subsistema A]

[Subsistema B]

[Subsistema C]